

## Content Approval Workflow

Magnolia (DX Core) comes with a four-eye content approval workflow configured on the Pages app.

For custom content-apps, the publish/unpublish configuration (availability, type, dialog, label, etc) is defined within the [actions definition](#) and the [action bar definition](#)

## Developer Side

### Custom App

For a Magnolia content app, add four-eyes workflow by replacing direct publish/unpublish actions with workflow dialog actions.

```
1 subApps:
2   browser:
3     actions:
4       activate: !override
5         $type: openDialogAction
6         dialogId: workflow-pages:publish
7         icon: icon-publish
8         availability:
9           rules:
10            notDeleted: &notDeleted
11              $type: jcrIsDeletedRule
12              negate: true
13            isPublishable: &isPublishable
14              $type: jcrPublishableRule
15      activateRecursive: !override
16        label: Publish incl. subnodes
17        $type: openDialogAction
18        dialogId: workflow-pages:publishRecursive
19        icon: icon-publish-incl-sub
20        availability:
21          rules:
22            notDeleted: *notDeleted
23            hasSubPages:
24              $type: jcrHasChildrenRule
25            nodeTypes:
26              - mgnl:folder
27              - mgnl:card
28            isPublishable: *isPublishable
29      deactivate: !override
30        $type: openDialogAction
31        dialogId: workflow-pages:unPublish
32        icon: icon-unpublish
33        availability:
34          rules:
35            notDeleted: *notDeleted
36            isPublished:
37              $type: jcrPublishedRule
```

## Out-of-the-Box App

For an existing OOTB app, use a decoration file instead. For example, to enable the workflow for the assets app.

```
1 # light-modules/spg-decorated/decorations/dam-app/apps/dam.yaml
2 subApps:
3   jcrBrowser:
4     actions:
5       publish: !override
6         $type: openDialogAction
7         dialogId: workflow-pages:publish
8         icon: icon-publish
9         availability:
10           rules:
11             notDeleted: &notDeleted
12             $type: jcrIsDeletedRule
13             negate: true
14             isPublishable: &isPublishable
15             $type: jcrPublishableRule
16
17       publishRecursive: !override
18         $type: openDialogAction
19         dialogId: workflow-pages:publishRecursive
20         icon: icon-publish-incl-sub
21         label: Publish with subpages
22         availability:
23           rules:
24             notDeleted: *notDeleted
25             hasSubPages:
26               $type: jcrHasChildrenRule
27               nodeTypes:
28                 - mgnl:folder
29                 - mgnl:asset
30             isPublishable: *isPublishable
31
32       unpublish: !override &unpublish
33         $type: openDialogAction
34         dialogId: workflow-pages:unPublish
35         icon: icon-unpublish
36         availability:
37           rules:
38             notDeleted: *notDeleted
39             isPublished:
40               $type: jcrPublishedRule
41
42       publishDeletion: !override
43         $type: openDialogAction
44         dialogId: workflow-pages:publishDeletion
45         icon: icon-delete
46         availability:
47           rules:
48             isDeleted:
49               $type: jcrIsDeletedRule
50
```

- ❗ Use the `module.yaml` file to ensure that the decoration is loaded after all other modules; otherwise, it may cause conflicts or overrides.

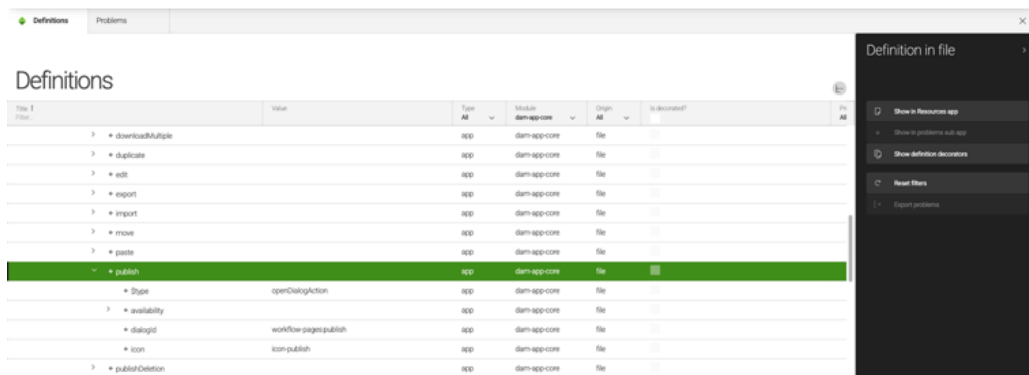
```
1 # light-modules/spg-decorated/module.yaml
2 version: 1.0
3 dependencies:
4   core:
```

```

5 version: 6/*
6 workflow-pages:
7   version: 6/*

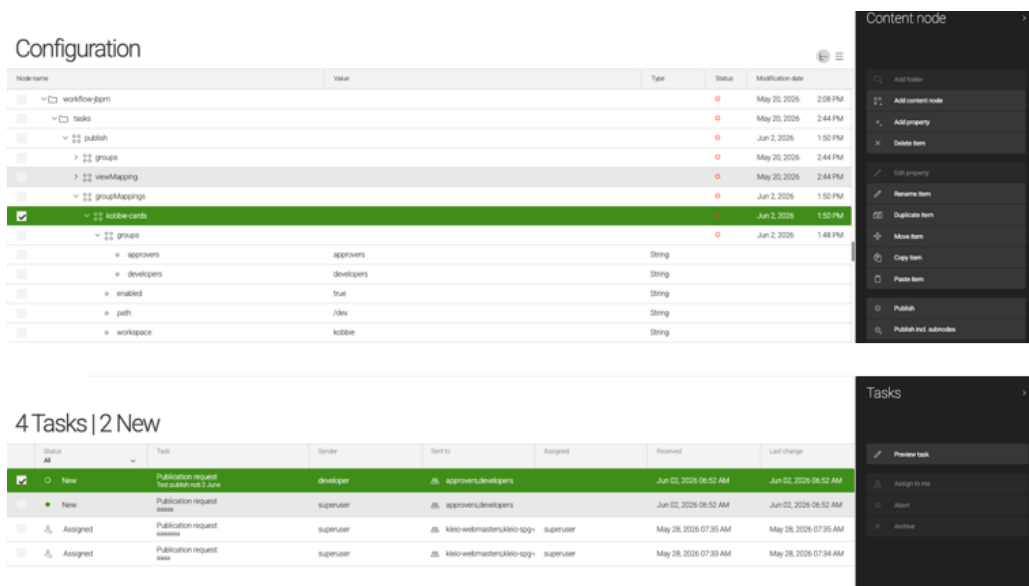
```

The decoration can be verified using the Definitions app.



## Approver Side

Create a new group mapping at `/modules/workflow-jbpm/tasks/publish/groupMappings` in the Configuration app. New publish events will be notified to all these groups.



To allow a role to claim the task for further publishing, quick aborting or archiving, configure permissions in the task app.

```

1 # light-modules/spg-decorated/decorations/tasks-app/apps/tasks-
  app.yaml
2 subApps:
3   browser:
4     actions:
5       claim:
6         availability:
7         access:
8         roles:

```

```

9         - approver
10        - superuser
11    abort:
12        availability:
13        access:
14        roles:
15            - approver
16            - developer
17            - superuser

```

To configure permissions on the Task Preview (Publication Request) screen for actual Publish, Reject, or Abort actions, decorate the workflow module:

```

1  # light-modules/spg-
   decorated/decorations/workflow/messageViews/publish.yaml
2  # light-modules/spg-
   decorated/decorations/workflow/messageViews/unpublish.yaml
3  actions:
4      review:
5          availability:
6          access:
7          roles:
8              - superuser
9              - approver
10     claim:
11         availability:
12         access:
13         roles:
14             - superuser
15             - approver
16     approve:
17         availability:
18         access:
19         roles:
20             - superuser
21             - approver
22     confirmRejection:
23         availability:
24         access:
25         roles:
26             - superuser
27             - approver
28     retry:
29         availability:
30         access:
31         roles:
32             - superuser
33             - approver
34     abort:
35         availability:
36         access:
37         roles:
38             - superuser
39             - approver
40             - developer

```

## Appendix: App Visibility

To allow a role to see a panel (a group of apps), configure the permission at

```
light-modules/spg-decorated/decorations/admincentral/config.yaml
```

```

1 groups:
2   - name: kobbie
3     label: Kobbie's Apps
4     apps:
5       - name: cards-app
6     permissions:
7       roles:
8         superuser: superuser
9         developer: developer
10        approver: approver
11

```

To allow a role to see an app, configure the app permission

```

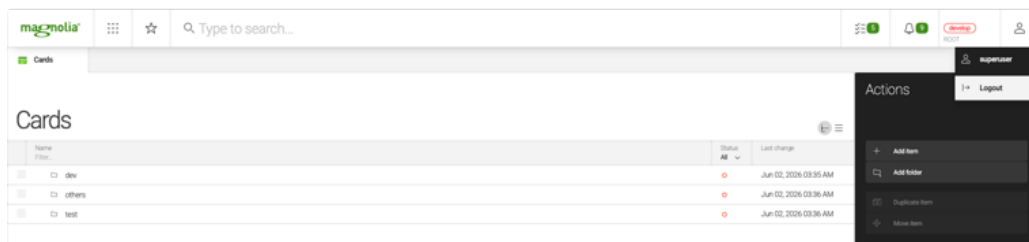
1 # light-modules/kobbie/apps/cards-app.yaml
2 !content-type:card
3 name: cards-app
4 label: Cards
5 icon: icon-templating-app
6 permissions:
7   roles:
8     - superuser
9     - developer
10    - approver

```

To allow a role to read/write in an app, configure the Role ACLs in the Security app. This rule allows the `developer` role to read and write in the `/dev` folder and all of its child items.

**i** The workspace name is defined in the Content Type YAML file.

From the superuser perspective, the cards app looks like this.



But on the developer side, only the dev folder is shown.

